

Three changes, planned before a line is written

Trade allocation as a switch · one press to confirm · pins on the photos carrying more than one finding
Rev 2 adds **Annex A**: the viewer, the pins and the AI we could clone — and the ones we already own

ISSUED	OWNER	PRODUCT	REPOSITORY
7 Oct 2026	D. Hopwood	instructBrain	d930b4b on main · CI green

Nothing in here is built yet. This is the plan for the three jobs you set, in the order you set them, with what each one touches, what it costs, and the four decisions I need from you before I start. Everything below was measured against your live database this evening, not guessed.

1 The three jobs, and what each one is really about

#	JOB	SIZE	WHY IT MATTERS	NEEDS YOU
1	Trade allocation becomes a switch	1 session	Today the trade layer is welded into the review screen, the publish gate, the distribution screen and the PDF whether you use it or not. Across 28 reports, nobody has filled in a directory template, and one trade link has ever been issued.	DECISION 1
2	One press to confirm everything	1.5 to 2 sessions	222 findings, 183 of them confirmed one press at a time, through five separate confirmation surfaces that each write their own row. This is the daily friction.	DECISION 2
3	Pins on a photo with several findings	1 to 2 sessions	25 of your photographs carry more than one finding, and each finding renders its own copy of the whole picture. The AI already knows which patch of the image it looked at, and that patch is thrown away every time.	DECISION 3

2 What I measured in your own data — 7 October 2026, live

MEASURE	NUMBER	WHAT IT TELLS US
Findings on the account	222	The unit of work in the review screen.
Confirmed by a person	183 (82%)	Confirmation is happening, by hand, one item at a time.
Findings with a trade assigned	51	The AI suggested a trade on 126. Only 51 have been settled by a person.
Findings carrying the AI's image region	105 of 222 (47%)	The model already returns the patch of the photograph it read. It is stored and never used.
Saved regions on the photo links	0	This is the gap. The drawing code looks at the photo link, the region sits on the finding, so no box is ever drawn.
Photographs	202	Photos read by the AI.
Photos carrying more than one finding	25	25 chances of the same picture appearing two, three, six times.
Hand-drawn markups on photos	0	The markup tool exists and has never been used. It is not the answer on its own.
Reports	28	13 published, 15 draft.
Project directory entries / contacts	2 / 2	The trade feature has effectively never been set up.
Directory templates / trade links issued	0 / 1	One subbie link in the life of the product.

Read together: the trade layer costs every user a screen, a gate and a set of buttons, and almost nobody uses it. The confirmation layer is used by everyone, every day, the hard way. The region data that would fix the photo problem is already being paid for and binned.

3 Job 1 — trade allocation as a switch

You asked for the whole feature to be an option: enable or disable. Here is what “the whole feature” actually is, because it is wider than one screen.

REVIEW	The Trade card and the bulk Confirm all trades action appear whenever the template says trades are required. The template flag is frozen into each survey definition, so it is per-template and not something you can turn off from the screen.
PUBLISH GATE	Publishing is blocked while any finding has no trade, on templates that require one. That gate lives in <code>issueBlockers()</code> in <code>src/lib/report/document.ts</code> and is read by the report screen and the publish button.
DIRECTORIES	Two screens: your organisation directory in Settings, and a project directory per project, each with contacts. 857 lines in the project one.
DISTRIBUTION	The send screen groups items by trade and fires one email per trade with its own extract and its own live link. With no trades assigned, it collapses to one email — which is what happened on the test send.
TRADE PORTAL	A public token page (<code>/trade/<token></code>) where a subbie marks their own items done. One has ever been issued.

OFF MEANS	• No trade card and no trade gate in the review screen — the finding card is shorter and nothing asks you for a trade. • Publishing no longer blocked on a missing trade. Everything else about the gate is unchanged. • The organisation directory and project directory screens are hidden from navigation. • Distribution shows one whole-report row: no per-trade grouping, no per-trade extracts. • The per-trade extract route and the trade portal go quiet; existing links are honoured read-only, nothing breaks for a subbie holding one. • The PDF drops its per-trade appendix. The report itself is untouched. • Emails still work end to end — the whole-report send is unaffected.
OFF NEVER MEANS	Deleting anything. Directory rows, contacts and assigned trades stay in the database and come back the moment the switch goes on. No published version is altered, and an already-published report always reads the same as the day it was published.
WHERE IT LIVES	A setting in Settings → Organisation, plus a per-report override, so “this job doesn’t need trades” is a switch on the day and not a global decision. Two new database columns, one migration, additive only.
WHY THAT SHAPE	Most of the machinery already exists as a per-template flag. What is missing is a control a person can reach. Per-template alone (today’s behaviour) means you cannot turn it off without a code change; organisation-only means one report cannot differ from the next.

Decision 1 — how far does the switch reach?

- A. Organisation setting plus a per-report override.

MY RECOMMENDATION

Default on for existing accounts so nothing changes for anyone until they switch it off, one control on the report for a job that does not need trades. Two columns, one migration, the honest cost is about a session including tests.
- B. Per report only.

Smaller and safer: the switch travels with the job. But you would still see the directory screens and the trade plumbing on every other report.
- C. Organisation only.

Simplest mental model, one switch for the account. The cost is that a one-off custom report cannot opt out while the account stays on.
- D. Leave it as it is.

Nothing changes. Honest answer: the trade layer is nearly unused, so this keeps costing every report a screen and a gate for the customers who do not need it.

4 Job 2 — one press to confirm everything

This is the one you called the big thing, and I agree: it is not a button problem, it is a “how many decisions does a person make per report” problem.

1 How it works today — five ways to say yes

MEASURED 1106-line screen

THE FIVE

- **Confirm** on a finding — accepts the AI's wording and status, one item at a time.
- **Confirm grade** on the condition card — a second, separate press for the same item.
- **Confirm assignment** on the trade card — a third press for the same item.
- **Confirm all trades 80% or over** — a bulk action that only covers trades the AI was confident about.
- **Confirm all findings** — a bulk action, but it skips exactly the items with the missing trades that the publish gate is waiting for, so you can confirm everything and still be blocked.

AND THEN

The draft summary has its own confirmation, and the reading view carries its own **Confirm this finding**. Six places in total, four of which are asking the same question at different levels.

WHAT GOES WRONG

You press Confirm all findings, the count says everything is accepted, and publishing still refuses because a trade is missing. The number on the button and the number that decides publishing are not the same number.

2 The proposal — one card, one button, one list of exceptions

PROPOSED one screen change

THE CARD

At the top of the review screen, one line and one button: **"222 findings drafted, 183 accepted, 39 need you"** and **Accept all 39**. That single press settles status, grade, trade and wording for every item the AI was confident about, in one write.

THE LIST

Under the button, only the exceptions — the items the AI was unsure about, the ones it could not assess, the ones where two items were found in one photo. Each has one press to settle, and the reason is written beside it in plain words.

THE ONE RULE

The number on the button always equals what will actually be written, and the same number decides publishing. No more "confirmed everything, still blocked".

PER ITEM

One button per finding instead of three: accepting a card settles its grade and its trade with the text. One undo, not three.

How it stays honest

- **Nothing is accepted that a person has not seen.** The card states the number it is about to accept, before it is pressed, and the number is what gets written.
- **Not assessed items are never swept in.** That rule exists today and stays: an item the AI could not read needs a human status, and one press cannot invent it.
- **The audit trail is unchanged.** Every accepted finding still records who confirmed it and when, exactly as it does now — so the accountability the reports rely on is identical.
- **The AI's confidence decides what goes in the quick pile.** Three confidence scores are already stored per finding (wording, grade, trade). Below the line? It goes on the list, not in the bulk accept.
- **One undo.** Published versions already freeze a copy, so a mis-press can never reach a published report.

Decision 2 — what does "accept all" include?

A. Accept everything the AI was confident about, list the rest. MY RECOMMENDATION

One press for the bulk, a short list of genuine judgement calls. On your current data that is roughly four in five findings settled in one press, with the doubtful fifth in front of you.

B. Accept absolutely everything, flag the doubtful ones afterwards. Fewest clicks, fastest, and it puts AI guesses into a report you sign. I would not recommend it on a document that goes to a client.

C. Keep per-item confirmation, but make it one press per item. A honest improvement, about half the work, but it does not fix the thing you actually described.

5 Job 3 — the photo carrying six findings

You asked whether we could enlarge the photo, drop a numbered pin per item and reference the pin number. Yes — and the groundwork is closer than you would think.

1 Why it repeats the picture today

MEASURED root cause

WHAT YOU SEE

25 of your photos carry more than one finding, and each finding draws its own copy of the whole photograph. A wall with four defects appears four times, full width, and nothing on the picture says which defect is which.

WHY

The code that draws a box round the relevant part of a photo reads the region saved on the finding's **photo link**. That column has **0 rows**. The AI's region is saved on the finding instead, in a different column, and nothing reads it there.

SO

The feature is half built and switched off by a wiring mistake. The box, the overlay and the numbered marker tool all already exist. 105 of your findings are carrying a region nobody has ever seen.

YOUR IDEA	One photo, numbered pins. The photograph is shown once. Every item the AI found on it gets a numbered pin at the place it was read from, and the item list underneath reads “Pin 3 · F-017 · cracked render to north elevation”. The number is the same on screen, in the PDF, on the shared page and in the trade extract.
1. ZOOM	Tapping a pin (or its item) zooms that patch full width. On paper, a small inset of the patch is printed beside the item instead of the whole photograph again — that is what takes pages out of the PDF.
2. ONLY YOUR PINS	In a trade extract, that trade’s pins stay bright and the others dim. A subbie holding a 40-item report sees the three things that are his, on the right photo.
3. PINS YOU CAN MOVE	The region is the model’s estimate, so it will sometimes be in the wrong place. A person drags the pin; that becomes a saved markup on the photo, so the correction survives into the published version. This is why the drag ships with the pins, not after them.
4. HONEST ABSENCE	117 findings have no region at all. Where the AI gave nothing, no pin is invented — the item appears in the list without a marker, and the photo is shown once. I would rather show nothing than point confidently at the wrong place on a client report.

Is there anything better than pins? Two things.

- **Auto-crop beats a pin on paper.** Because the AI already knows the patch, the PDF can print a tight crop of the defect instead of a wide site photo where the problem is 3% of the frame. A pin says where to look; a crop means you do not have to look. I would do both: pins on the photo, crops in the printed item list.
- **One photo instead of six is the real win.** Not just tidier — it cuts the report by roughly a page per multi-item photograph and makes a snagging report readable on site, on a phone, in the rain.

Decision 3 — pins everywhere, or only where they earn their place?

A. Pins only where a photo carries more than one finding. MY RECOMMENDATION

A single-finding photo stays exactly as it looks today, big and clear. The new treatment appears only where the problem is. Nothing you like changes.

B. Pins on every photo. More consistent, and it costs you the large single photograph that reads best on a phone.

C. Region wiring first, pins later. Ship the wiring alone (a small job, half a session) so the boxes appear in reports tomorrow, and decide the pins after you have looked at a real one.

6 Order, size, and what I would do first

ORDER	JOB	MY TIME	RISK	NOTES
1	Trade allocation switch	1 session	Low. Additive database change, one gate to re-read, surfaces to hide.	Do this first as you asked. It removes work from the other two — with trades off, the review card loses a whole row of buttons.
2	One press to confirm	1.5 to 2 sessions	Medium. It rewrites the busiest screen in the product, and every write path is covered by tests that will have to move with it.	The biggest job, the biggest payoff. I would build the exception list first, then the button, so the screen is never in a half-state.
3	Region wiring	half a session	Low. Save the region the AI already returns, let the drawing code find it.	Could slot in early as a quick visible win — say the word and I will move it forward.
4	Pins, zoom, crops	1 to 1.5 sessions	Medium. New drawing in three places (screen, PDF, shared page) that must agree.	Depends on 3. Drag-to-correct is part of this job, not a follow-up. Annex A removes the design work from this one — the pin mechanics and palette already exist in instructSite — but not the verification, so the estimate stands as the worst case.

A “session” is one working stretch of mine. Every one of them lands the same way: tests, lint, build, a push, CI green, and a re-walk of the live screens after you publish.

7 What I need from you

#	DECISION	MY RECOMMENDATION
1	How far does the trade switch reach?	A — organisation setting plus a per-report override.
2	What does “accept all” include?	A — everything the AI was confident about, doubtful items listed for you.
3	Pins everywhere or only on multi-finding photos?	A — only where a photo carries more than one finding.
4	Do you want the region wiring pulled forward as a quick win?	Your call. It is half a session and it makes boxes appear in reports immediately.
5	Do we build the pins from your own instructSite parts and keep the AI on Gemini?	Yes to both — clone the pin canvas and palette, keep Gemini, revisit SAM only after we have judged twenty real boxes.

Limits, stated plainly. Every number above was read from the live database on 7 October 2026 and will drift as you use the product. Times are my working sessions, not calendar days, and they include tests and verification. Nothing here is built: no code has been written for any of the three jobs. There is still **no full type check** in this project — my box cannot run one (the container is capped at 2 GB and `tsc` is killed) and the build workflow has no typecheck step, so I check the files I change and say so. And the AI's image region is an estimate, which is exactly why job 3 ships with the ability to drag a pin, not after it.

A Annex A — the viewer, the pins and the AI we could clone

You asked two things: whether we could use a PDF viewer like the one in instructSite, and what AI technology is out there that we could clone. I went through your own codebase first, then the market. The short version: **you already own most of this, and the AI part needs no new spend.**

A1 The viewer in instructSite — and the pins you already built		IN YOUR CODE	verified
PDF VIEWER	<code>src/components/tooling/PdfViewer.tsx</code> — 94 lines. Built on react-pdf 10.4.1 and pdfjs-dist 6.1.200 (Mozilla's engine), with page navigation, zoom, download, and the worker wired up properly. It renders straight from bytes already in memory. That is the whole viewer — small, and easy to lift.		
THE PINS	<code>src/components/project/DrawingCanvas.tsx</code> — 993 lines, and this is the part that matters for job 3. It already does pin-drop on a drawing : a pin carries an x and a y as a percentage of the image, there is a drop mode and a view mode, clicking a pin selects it, and the active pin is highlighted. That is exactly the mechanic we planned for findings on a photograph.		
THE COLOURS	<code>src/lib/pin-color.ts</code> — a curated 12-hue palette , colour-blind aware , derived from a single key (a trade package or a subcontractor) so that “pins, list rows and cockpit chips stay in sync”. Same idea we need: a finding's pin colour matching its item row.		
WHY IT IS CHEAP	Your instructBrain markup already stores positions the same way — x and y as a fraction of the image, 0 to 1. So this is a port, not a research project . The design decisions are made and the palette is chosen.		
HONEST NOTE	The PDF viewer is a convenience for reading a published report inside the app. It does nothing for the pin problem — photographs are not PDFs. I would clone the pin parts for job 3 and treat the PDF viewer as a separate, smaller decision.		
A2 The AI: you are already paying for the right one		NO NEW COST	one prompt change
WHAT RUNS TODAY	Vision is Gemini 2.5 Flash as the triage pass and Gemini 2.5 Pro as the escalation pass, Google direct (<code>src/lib/ai/config.ts</code>). That is how the 105 regions on your findings were produced.		
WHAT GEMINI CAN DO	Google's own API documentation covers points and 2D bounding boxes returned straight from the model, normalised 0–1000 in <code>[y_min, x_min, y_max, x_max]</code> form. No extra service, no extra vendor, no GPU.		
SO	Job 3 needs no new AI and no extra cost . We ask the model we already call for a box on every observation, keep it, and draw it. The 47% coverage we measured is a prompt and wiring gap, not a capability gap.		
A3 If we ever want better pointing than Gemini		OPEN MODELS	all verified 7 Oct
MOLMO — AI2	Pointing is its speciality. Apache 2.0 licence, open weights (7B and 72B), trained on a million curated image-text pairs. In the PointArena benchmarking, Molmo-7B-D beat Gemini-2.5-Flash by 196 Elo points on pointing — so “free and better at this one trick” is a real possibility. Caveat I would not hide: its model card says Apache 2.0 but “intended for research and educational use”, which is a question for a commercial product, and it needs a GPU to run.		
QWEN3-VL — ALIBABA	Open weights, dense and MoE sizes, and its own documentation claims stronger 2D grounding plus 3D position reasoning. Runs on vLLM or SGLang, or as a hosted API. Licence varies by size, so it is a per-model check before commercial use. Also needs a GPU if self-hosted.		
SAM 2 / SAM 3 — META	Apache 2.0 — I read the licence in the repository to be sure. It takes a point or a box and returns a precise outline rather than a rectangle. That is the difference between “a box roughly round the defect” and “the crack outlined exactly”, which would look superb in a client report. Needs a GPU, or a paid per-image API.		
FLORENCE-2 — MICROSOFT	Small (0.77B) and does detection and segmentation from a text prompt, which makes it cheap to run. Its licence was not verified in this pass, so treat it as a lead rather than a decision.		

A4 Web libraries worth standing on		LICENCES CHECKED 4 options
PDF.JS / REACT-PDF	Apache 2.0, Mozilla. Already in instructSite, so it is proven on your own stack. The default answer for viewing a report in the browser.	
OPENSEADRAGON 6.1.1	New BSD licence. A high-resolution zoomable image viewer for desktop and mobile, and it is the thing that makes “tap the pin, the photo zooms to that patch” feel right on a phone. Worth it only if our own zoom feels sluggish on full-size site photos.	
ANNOTORIOUS	BSD 3-Clause, with official React bindings, and it works on top of OpenSeadragon. It is the closest thing to a drop-in for drawing and editing regions on an image — which is what a person does when they drag a pin to the right place.	
FRESH-AIR-PDF	An open-source PDF.js-based React viewer with an annotation toolbar, form fields, signatures and JSON import and export. Version 0.1.5, six releases, no dependents — interesting, too young to build a product on. Filed for later.	

What I recommend, in three tiers

- Tier 1 — no new technology, no new spend. DO THIS

Ask the Gemini model we already call for a box on every observation, keep it, draw it on the photo as a numbered pin, and take the pin mechanics and the colour palette from instructSite. Job 3 then becomes: wiring, a port, and the PDF-side crops.
- Tier 2 — small, if the photos feel heavy on a phone. Add tap-to-zoom and pinch-zoom using the pattern already in your own DrawingCanvas, and move to OpenSeadragon only if that is not enough.
- Tier 3 — deferred, and it has a cost. SAM 2 behind each box for exact outlines, and a pointing specialist like Molmo or Qwen3-VL if Gemini’s boxes prove too loose on real site photos. Both need GPU hosting — I would price that rather than guess, and I would not spend it until we have looked at twenty real boxes and decided they are not good enough. My recommendation: look first, spend later.

Decision 5 — and it is the only new one

- Do we build the pins from your own instructSite parts, and keep the AI on Gemini?

My recommendation: yes to both. Clone the pin canvas and the colour palette, keep Gemini for the boxes, and revisit SAM only after we have judged twenty real ones. Nothing in Tier 3 changes what a user sees today.

B Sources — everything in Annex A, and where it came from

CLAIM	SOURCE
instructSite viewer and pin system	is-repo : src/components/tooling/PdfViewer.tsx , src/components/project/DrawingCanvas.tsx , src/lib/pin-color.ts , package.json (react-pdf 10.4.1, pdfjs-dist 6.1.200).
instructBrain runs Gemini 2.5 Flash and Pro	ib-repo : src/lib/ai/config.ts lines 193–194.
Gemini returns points and 2D bounding boxes	ai.google.dev — Gemini API, spatial reasoning; and the Google Cloud bounding-box detection page ([y_min, x_min, y_max, x_max] , normalised 0–1000).
Molmo is Apache 2.0, research and educational use noted	huggingface.co/allenai/Molmo-7B-O-0924 — model card, licence and use section.
Molmo-7B-D beat Gemini-2.5-Flash by 196 Elo on pointing	PointArena, arXiv 2505.09990.
Qwen3-VL does stronger 2D grounding	github.com/QwenLM/Qwen3-VL — key enhancements; technical report arXiv 2511.21631.
SAM 2 is Apache 2.0	github.com/facebookresearch/sam2 — LICENSE file, read directly.
OpenSeadragon is New BSD; Annotorious is BSD 3-Clause	openseadragon.github.io licence section; github.com/annotorious/annotorious licence section.
fresh-air-pdf maturity	npmjs.com/package/fresh-air-pdf — version 0.1.5, six versions, zero dependents.

Two things I did not verify and am therefore not claiming: Florence-2’s licence, and the licence per Qwen3-VL model size. Both are marked above as checks outstanding rather than facts.